



Università degli Studi di Salerno

Dipartimento di Informatica

Corso di Ingegneria del Software: Tecniche Avanzate

CodeSmile



Pre Modification Test Incident Document
Versione 1.0

Team

Choaib Goumri NF22500223
Mattia Gallucci NF22500127

Repository GitHub

Anno Accademico 2025–2026

Indice

1	Introduzione	1
2	Incident Report	2
2.1	Test Incident 1 – TI1	2

1 Introduzione

Questo elaborato espone gli esiti delle procedure di verifica preliminari applicate al progetto CodeSmile, condotte anteriormente all'implementazione delle modifiche pianificate. L'indagine copre in modo trasversale le criticità emerse, intercettando sia i malfunzionamenti intrinseci del software sia le incongruenze nella suite di test, analizzandole a ogni livello di granularità.

La finalità primaria del report è solidificare le deviazioni osservate rispetto al oracolo, offrendo una disamina tecnica rigorosa che distingue tra errori applicativi e falsi positivi derivanti da test errati.

Per ogni incidente censito, vengono delineate le cause radice e le azioni correttive intraprese, con l'obiettivo ultimo di consolidare l'affidabilità dell'architettura e raffinare la pipeline di validazione del software.

2 Incident Report

2.1 Test Incident 1 – TI1

Informazioni generali

- **Nome:** TI1
- **Ambiente:** Ambiente di Sviluppo Locale (Windows)
- **Tipologia di test:** Test di unità
- **File:** `test/unit_testing/components/test_project_analyzer.py`
- **Metodo:** `test_analyze_project_empty_directory`

Analisi del Problema

Descrizione e Causa:

Il test `test_analyze_project_empty_directory` era stato progettato con l'assunzione che l'analisi di una cartella vuota dovesse terminare con successo (ad esempio, restituendo 0 code smells). Tuttavia, il design del componente `ProjectAnalyzer` implementa un approccio difensivo ("fail-fast"): esso solleva esplicitamente un'eccezione `ValueError` qualora non vengano individuati file sorgente validi. Questa discrepanza tra l'oracolo del test (che non prevedeva eccezioni) e l'implementazione reale causava il fallimento sistematico della suite con l'errore `ValueError: The project ... contains no Python files`.

Riproduzione: Eseguire il comando:

```
pytest test/unit_testing/components/test_project_analyzer.py
::test_analyze_project_empty_directory
```

Risoluzione e Scelte Tecniche

- **Stato:** Risolto.
- **Componenti coinvolte:** `ProjectAnalyzer` e relativo test unitario.
- **Strategia di Risoluzione:** Si è scelto di non modificare il codice di produzione (che agisce correttamente segnalando l'anomalia di un progetto vuoto), ma di correggere l'aspettativa del test.

- **Soluzione Tecnica:** Il test è stato riscritto utilizzando il context manager `pytest.raises(ValueError)`. Questo costrutto permette di asserire formalmente che, date le precondizioni (directory vuota), il sollevamento dell'eccezione è il comportamento *corretto* e atteso. È stato inoltre aggiunto il controllo sul messaggio di errore tramite il parametro `match` per garantire che l'eccezione sia esattamente quella prevista.